

Dynamic Programming or DP

Updated : 26 Jan, 2026



Dynamic Programming is an algorithmic technique with the following properties.

It is mainly an optimization over plain recursion. Wherever we see a recursive solution that has repeated calls for same inputs, we can optimize it using Dynamic Programming.

The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed again. This simple optimization typically reduces time complexities from exponential to polynomial.

Some popular problems solved using Dynamic Programming are [Fibonacci Numbers](#), [Longest Common Subsequence](#), [Bellman-Ford Shortest Path](#), [Floyd Warshall](#), [Edit Distance](#) and [Matrix Chain Multiplication](#).

Dynamic Programming: Linear

Fibonacci sequence is optimized using the tabulation (bottom-up) approach, which reduces the time complexity to linear.

fib[] =

0	1	1	2	1+2	
0	1	2	3	4	5

```
fib[0] = 0;
fib[1] = 1;
for (i = 2; i <= n; i++) {
    fib[i] = fib[i-1] + fib[i-2];
}
return fib[n];
```

Dynamic Programming or DP

< || >

2 / 2

Types of DP

[Recursion](#)

[Tabulation vs Memoization](#)

[How to solve a DP Problem](#)

Problems

[Fibonacci numbers](#)

[Fibonacci Numbers](#)

[Fibonacci Numbers](#)

[Climbing Stairs](#)

[Climbing Stairs with 3 Moves](#)

[Climbing Stairs](#)

[Sum Segments](#)

[atalan Number](#)

[Unique BSTs](#)

[Valid Parenthesis](#)

[to Triangulate a Polygon](#)

[um in a Triangle](#)

[um Perfect Squares](#)

[to Partition a Set](#)

[ial Coefficient](#)

[l's Triangle](#)

[ow of Pascal Triangle](#)

[um in a Triangle](#)

problems

[Robber](#)

[ost Path](#)

[le Ways](#)

[t Sum Problem](#)

[hange problem - Count Ways](#)

[hange – Minimum Coins to Make Sum](#)

[ng Fence Algorithm](#)

[g a Rod](#)

[Game](#)

[st Common Substring](#)

[all paths in a Grid](#)

[in a Grid with Obstacles](#)

[utations with K Inversions](#)

[v's using Special Keyboard](#)

n Problems

[Overflow](#)

[st Common Subsequence](#)

[st Increasing Subsequence](#)

[istance](#)

[st Divisible Subset](#)

[ted Job Scheduling](#)

[napsack Problem](#)

[ng Items in 0/1 Knapsack](#)

[nded Knapsack](#)

[Break Problem](#)

[tacking Problem](#)
[tacking Problem](#)
[on Problem](#)
[st Palindromic Subsequence](#)
[st Common Increasing Subsequence \(LCS + LIS\)](#)
[stinct subset \(or subsequence\) sums](#)
[Derangements](#)
[um insertions for palindrome](#)
[ard Pattern Matching](#)
[ar Expression Matching](#)
[ge Balls with adjacent of different types](#)
[st Subsequence with 1 adjacent difference](#)
[um size square sub-matrix with all 1s](#)
[an-Ford Algorithm](#)
[Warshall Algorithm](#)
[um Tip Calculator](#)

Problems

[st X Bordered Square](#)
[ropping Problem](#)
[lrome Partitioning](#)
[lromic Substring Count](#)
[Wrap Problem](#)
[al Strategy for a Game](#)
[ainter's partition problem](#)
[am for Bridge and Torch problem](#)
[Chain Multiplication](#)
[ig Matrix Chain Multiplication](#)
[um sum rectangle](#)
[Buy and Sell - At-Most k Times](#)
[Buy and Sell - At Most 2 Times](#)
[st to sort strings using Reversals](#)
[of AP Subsequences](#)
[Trees](#)
[height of Tree when any Node can be Root](#)
[st repeating and non-overlapping substring](#)
[lrome Substrings Count](#)

ms Sorted by Topic / Dimensions

[andard Problems and Variations.](#)
[blems Dimension Wise \(1D, 2D and 3D\).](#)

Problems Topic Wise

ced Concepts

sking and DP1

ing Salesman Problem

DP

over Subsets

_inks:

Tutorial

Interview Questions

ce Dynamic Programming

on Dynamic Programming

Introduction to DP

[Visit Course ↗](#)

••

Competitive Programming

ulation

Do Not Sell or Share My Personal Information